ICT203 Artificial Intelligence and Intelligent Agents
Assignment 1 Report

Name: Styverson Ng
Kaplan Student Number: CT0372348
Murdoch Student Number: 35427675

Table of Contents

Problem Description 3
Environment 3
Conditions 3
Rewards 3
Components of Search Problem: 4
Algorithm Designs and Functions 5
Search Algorithms 5
Depth-First Search 6
Breadth-First Search 8
A* Search (Heuristics) 10
Results and Findings 11
Algorithm Performance 11
Practical Implications 12
Limitations 13
User Guide 14
Software Requirements 14

Problem Description

Environment

This reports denotes the development of an intelligent agent capable of navigating a taxi within a given environment (Gymnasium) tasked to pick up and drop off passengers. By utilizing search algorithms, this intelligence agent searches for the shortest path possible to complete its assigned task, whilst maximizing the rewards.

Conditions

- **Grid Size:** 5x5 grid, 25 possible locations
- **Taxi States:** 25 positions
  Passenger States: 5 possible states (Red, Green, Yellow, Blue, In Taxi):
  0 = Red, 1 = Green, 2 = Yellow, 3 = Blue, 4 = In-Taxi
- **Drop-off Locations:** 4 designated drop-off/pick-up locations (Red, Green, Yellow, Blue)
  0 = Red, 1 = Green, 2 = Yellow, 3 = Blue 1 = Green
- **Actions:** The taxi is capable of performing 6 actions:

  0 = Move South (down)

  1 = Move North (up)

  2 = Move East (right)

  3 = Move West (left)

  4 = Pick up passenger

  5 = Drop off passenger

Rewards

- **-1 point:** For each move action, unless another reward is triggered. This encourages the taxi to find the shortest path.
- **-10 points:** For illegal pickup or dropoff actions. This penalizes the taxi for trying to pick up a passenger where there isn't one or dropping off a passenger at the wrong location.
- **+20 points:** For a successful dropoff. This is the primary goal of the task.

Components of Search Problem:

Initial State:
The initial state is defined by the combination of the taxi's location and the passenger's designated pick-up point. This state serves as the starting configuration for the search process.

Goal State:
The goal state is achieved when the passenger is successfully dropped off at the specified destination. This marks the completion of the task and signifies the successful termination of the search.

Actions:
The taxi operates within a grid-based environment and is capable of executing the following actions:

- **Move** in any of the four cardinal directions (north, south, east, west).
- **Pick up** the passenger from the designated pick-up location.
- **Drop off** the passenger at the specified drop-off point.

Transition Model:
The transition model defines how the taxi transitions from one state to another through the execution of an action. Each action leads to a deterministic change in the state:

- **Movement actions** (e.g., moving west) increment the state index according to the grid structure.
- **Passenger pick-up and drop-off** actions alter the passenger's status (either in the taxi or at a location) and modify the state accordingly. For instance, moving west may increment the state by 3, while a successful drop-off increments the state by 5.

Reward Function:
The reward function is designed to encourage efficient and accurate pathfinding while penalizing inefficient or invalid moves:

- Each **movement** incurs a penalty of **-1** to discourage unnecessary steps.
- **Illegal pick-up or drop-off** actions result in a penalty of **-10** to enforce adherence to correct procedures.
- A **successful drop-off** yields a reward of **+20** to incentivize achieving the goal efficiently.

Algorithm Designs and Functions

Search Algorithms

- Depth-First Search
- Breadth-First Search
- A* Search

This combination of search algorithms allows for a comprehensive analysis of pathfinding in the Taxi environment. Depth-First Search (DFS), which explores each branch to its deepest point before backtracking, serves as a benchmark for comparison. Its selection as a constant in this analysis stems from its relatively balanced performance profile, lacking significant strengths or limitations. This provides a neutral baseline against which to assess the performance of the other algorithms.

Breadth-First Search (BFS), on the other hand, explores the search space layer by layer, guaranteeing the shortest path to a goal state. This characteristic makes BFS a valuable point of reference, particularly for understanding the efficiency and optimality of the other search algorithms. However, BFS's exhaustive exploration of each layer can lead to slower performance in complex or deep search spaces.

Finally, the A* search algorithm is included due to its informed nature. By incorporating a heuristic function, A* can prioritize the exploration of more promising paths, potentially leading to faster and more efficient pathfinding compared to uninformed methods like BFS and DFS. The choice of A* allows for the evaluation of the impact of heuristics on search performance in the Taxi domain.

Depth-First Search

```
PROCEDURE depth_first_search
    Initialize a timer to track the duration
    Set the initial step count to 0
    Set the initial reward value to 0

    Create a copy of the environment to avoid altering the original
    Reset the environment copy and obtain the initial state

    Create a stack to store states and their corresponding paths
    Add the initial state and an empty path to the stack
    Create a set to track visited states
    Create a dictionary to store the predecessors of visited states
    Mark the initial state as visited

    While the stack is not empty:
        Remove the top state and path from the stack

        For each possible action from the current state:
            Simulate taking the action in the environment copy
            Observe the resulting state, reward, and completion status

            Increment the step count
            Add the obtained reward to the total reward

            If the resulting state has not been visited:
                Mark the resulting state as visited
                Record the current state and action as the predecessor of the resulting state
                Add the resulting state and the updated path to the stack

            If the goal is reached (completion status is true):
                Reconstruct the path from the initial state to the goal state using the predecessor information
                Display the results, including the algorithm name, duration, step count, path, and total reward
                End the search

    If no solution is found, indicate that no path to the goal could be discovered
END PROCEDURE
```

The Depth-First Search (DFS) algorithm, as illustrated in the provided pseudocode, employs a systematic approach to explore the search space by prioritizing depth over breadth. Its logic is based on a stack data structure, which facilitates the "last-in, first-out" order of exploration. The algorithm begins by initializing a stack with the starting state and an empty path. It then iteratively explores deeper into the search tree by selecting a previously unvisited state from the stack and expanding its possible actions.

The flow of the DFS algorithm can be summarized as follows:

1. **Initialization:** Prepare the environment, create a stack, and mark the initial state as visited.
2. **State Selection:** Select a state from the stack.
3. **Action Expansion:** Explore each possible action from the selected state.
4. **New State Evaluation:** Evaluate the resulting state from each action, checking if it has been visited or if it is the goal state.

5. **State and Path Update:** If the resulting state is new, mark it as visited, update the path, and add it to the stack.
6. **Goal Check:** If the goal state is reached, reconstruct the path and display the results.
7. **Repeat:** Repeat steps 2-6 until the stack is empty or the goal is found.

The justification for using DFS in this context is rooted in its ability to function as a fundamental and foundational search algorithm. Its performance characteristics, in terms of speed, efficiency, and optimality, providing a baseline for comparison to better understand the behavior of more sophisticated algorithms. This makes the evaluation of other search algorithms more informative by providing a reference point for comparison.

By prioritizing depth, DFS can potentially find a solution quickly, especially if the goal state is located deep within the search tree. This characteristic makes DFS a valuable tool for scenarios where the depth of the solution is unknown or where a rapid solution is preferred, even if it is not guaranteed to be the shortest path.

However, it's important to acknowledge that DFS has limitations. Its exhaustive depth-first exploration can lead to it getting stuck in infinite loops or exploring unnecessary branches in complex search spaces.

Breadth-First Search

```
PROCEDURE breadth_first_search
    Initialize a timer to track the duration
    Set the initial step count to 0
    Set the initial reward value to 0

    Create a copy of the environment to avoid altering the original
    Reset the environment copy and obtain the initial state

    Create a queue to store states and their corresponding paths
    Add the initial state and an empty path to the queue
    Create a set to track visited states
    Create a dictionary to store the predecessors of visited states
    Mark the initial state as visited

    While the queue is not empty:
        Remove the first state and path from the queue

        For each possible action from the current state:
            Simulate taking the action in the environment copy
            Observe the resulting state, reward, and completion status

            Increment the step count

            If the resulting state has not been visited:
                Mark the resulting state as visited
                Add the resulting state and the updated path to the queue
                Record the current state and action as the predecessor of the resulting state
                Add the obtained reward to the total reward

            If the goal is reached (completion status is true):
                Reconstruct the path from the initial state to the goal state using the predecessor information
                Display the results, including the algorithm name, duration, step count, path, and total reward
                End the search

    If no solution is found, indicate that no path to the goal could be discovered
END PROCEDURE
```

The Breadth-First Search (BFS) algorithm adopts an approach that explores the search space by prioritizing breadth over depth. Its logic centers around a queue data structure, which enforces a "first-in, first-out" order of exploration. This ensures that the algorithm explores all immediate neighbors of a state before venturing deeper into the search tree.

The flow of BFS can be summarized as follows:

1. **Initialization:** Prepare the environment, establish a queue, and mark the initial state as visited.
2. **State Dequeue:** Remove the state at the front of the queue.
3. **Action Exploration:** Examine each possible action from the dequeued state.
4. **New State Evaluation:** Assess the resulting state from each action, checking if it has been visited or if it is the goal state.

5. **State and Path Enqueue:** If the resulting state is new, mark it as visited, update the path, and add it to the rear of the queue.
6. **Goal Check:** If the goal state is reached, reconstruct the path and display the results.
7. **Repeat:** Repeat steps 2-6 until the queue is empty or the goal is found.

The justification for using BFS in this context is rooted in its ability to guarantee the shortest path to the goal state. By exploring the search space layer by layer, BFS ensures that no potential shorter path is overlooked. This characteristic makes BFS a valuable tool for scenarios where optimality is paramount, such as finding the most efficient route in a navigation system or determining the shortest sequence of steps in a problem-solving task.

However, BFS is not without limitations. Its exhaustive breadth-first exploration can lead to high memory consumption and slower performance in large or complex search spaces, especially when the goal state is located deep within the tree.

A* Search (Heuristics)

```
FUNCTION heuristic(state)
        Decode state into taxi position, passenger location, and destination
        Calculate Manhattan distance from taxi to passenger (0 if passenger is in taxi)
        Calculate Manhattan distance from passenger (or taxi if passenger is in it) to destination
        Return the sum of the two distances
    END FUNCTION

PROCEDURE a_star_search
    Initialize a timer to track the duration
    Set the initial step count to 0
    Set the initial reward value to 0

    Create a copy of the environment to avoid altering the original
    Reset the environment copy and obtain the initial state

    Create a priority queue to store states, their costs, and corresponding paths, prioritized by the estimated total cost (cost + heuristic)
    Add the initial state with a cost of 0 and an empty path to the priority queue
    Create a set to track visited states
    Create a dictionary to store the predecessors of visited states
    Mark the initial state as visited
    Initialize a dictionary to store the cost to reach each state, starting with 0 for the initial state

    While the priority queue is not empty:
        Remove the state, current cost, and path with the lowest estimated total cost from the priority queue

        For each possible action from the current state:
            Simulate taking the action in the environment copy
            Observe the resulting state, reward, and completion status
            Calculate the cost to reach the new state as the current cost plus the cost of the action

            If the resulting state has not been visited or the calculated cost is lower than the previously recorded cost:
                Increment the step count
                Update the recorded cost for the resulting state
                Mark the resulting state as visited
                Record the current state and action as the predecessor of the resulting state
                Calculate the estimated total cost using the heuristic function
                Add the resulting state, cost, and updated path to the priority queue
                Add the obtained reward to the total reward

            If the goal is reached (completion status is true):
                Reconstruct the path from the initial state to the goal state using the predecessor information
                Display the results, including the algorithm name, duration, step count, path, and total reward
                End the search

    If no solution is found, indicate that no path to the goal could be discovered
END PROCEDURE
```

The A* Search algorithm is an informed search strategy that leverages a heuristic function to guide its exploration of the search space. This heuristic function estimates the cost from a given state to the goal state, enabling A* to prioritize paths that appear more promising. The algorithm employs a priority queue to manage states, where the priority is determined by the sum of the cost to reach a state and the heuristic estimate from that state to the goal.

The flow of A* Search can be summarized as follows:

1. **Initialization:** Prepare the environment, establish a priority queue, and initialize cost and visited state tracking.
2. **State Dequeue:** Remove the state with the lowest estimated total cost from the priority queue.
3. **Action Exploration:** Examine each possible action from the dequeued state.

4. **New State Evaluation:** Assess the resulting state from each action, calculating its cost and checking if it has been visited or if it is the goal state.
5. **Cost and Heuristic Calculation:** Calculate the cost to reach the new state and its heuristic estimate.
6. **State and Path Enqueue:** If the resulting state is new or has a lower cost than previously recorded, update the cost, mark it as visited, and add it to the priority queue based on its estimated total cost.
7. **Goal Check:** If the goal state is reached, reconstruct the path and display the results.
8. **Iteration:** Repeat steps 2-7 until the priority queue is empty or the goal is found.

The justification for using A* Search lies in its ability to efficiently explore the search space by prioritizing seemingly promising paths. By incorporating a heuristic, A* can potentially find a solution faster than uninformed search algorithms like BFS and DFS, especially in complex or large search spaces. This makes A* a valuable tool for scenarios where finding an optimal or near-optimal solution efficiently is crucial.

However, the effectiveness of A* heavily relies on the quality of the heuristic function. A poorly chosen heuristic can mislead the search and result in suboptimal performance.

Results and Findings

Algorithm	Average Duration (ms)	Average Total Steps	Average Path Length
DFS	0.012	113.126	11.493
BFS	0.018	124.491	11.013
A* Search	0.010	89.891	11.013

Algorithm Performance

The analysis of the three pathfinding algorithms within the Taxi-v3 environment reveals distinct performance characteristics. The A* search algorithm demonstrates the most efficient performance in terms of both duration and the number of steps taken. This efficiency can be attributed to its informed search strategy, which leverages a heuristic function to guide the exploration process and prioritize seemingly promising paths.

In contrast, the Breadth-First Search (BFS) algorithm, while guaranteeing the shortest path, demonstrates a significantly longer average duration and explores a wider range of states compared to A*. This observation underscores the inherent trade-off between optimality and computational expense in BFS, as its exhaustive layer-by-layer exploration ensures finding the shortest path but incurs a higher computational cost.

The Depth-First Search (DFS) algorithm, while faster than BFS, still yields comparatively longer paths. This presents a trade-off between speed and path optimality. While it is the fastest algorithm, it produces the least optimal paths. This suggests that DFS's depth-first exploration strategy, while potentially quick, can mislead into getting "trapped" in deep branches and missing shorter paths.

Practical Implications

The findings of this analysis offer valuable insights for practical applications. Firstly, the significant impact of the heuristic function on the performance of A* search emphasizes the importance of careful heuristic design in informed search strategies. A well-crafted heuristic can substantially enhance the efficiency of the search process, leading to faster and more effective solutions.

Secondly, the choice of algorithm should be guided by the specific requirements and constraints of the problem domain. If minimizing the time taken to find a solution is paramount, DFS might be preferred, whereas BFS is the optimal choice when the shortest path is critical, even at the expense of increased computation time. A* search presents a compelling alternative when a balance between speed and optimality is desired. These insights can inform algorithm selection for various pathfinding applications, depending on whether the priority lies in finding the fastest solution, the shortest path, or a compromise between the two.

Limitations

Heuristic Design for A* Search

The performance of the A* search algorithm is critically dependent on the quality of the heuristic function employed. A well-crafted heuristic can significantly enhance the algorithm's efficiency by effectively guiding the search process towards the goal state. In this study, the use of a rudimentary heuristic function resulted in A* search exhibiting a performance profile similar to that of Breadth-First Search (BFS), particularly in terms of the average path length. This observation suggests that the heuristic function employed did not adequately capture the problem's structure and failed to provide sufficient guidance to the search process. Consequently, A* search did not fully realize its potential for superior performance, as evidenced by its convergence with BFS in terms of path optimality.

The exploration of more sophisticated heuristics could potentially lead to substantial improvements in the efficiency of A* search. By employing a more informative heuristic, A* search could potentially achieve shorter paths and reduced computation time compared to BFS, demonstrating its full potential for efficient and optimal pathfinding. The development of effective heuristics is therefore crucial for maximizing the performance of A* search and realizing its benefits in various problem domains.

Comparative Analysis with Additional Algorithms

This study focused on three fundamental search algorithms: Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search. However, a more comprehensive evaluation could be achieved by including a comparative analysis with other relevant algorithms, such as Uniform Cost Search (UCS) and Iterative Deepening A* (IDA*). UCS is an uninformed search algorithm that guarantees finding the shortest path based on the cumulative cost of reaching a state, while IDA* is a space-efficient variant of A* that combines iterative deepening with A*'s heuristic guidance. Comparing the performance of these algorithms against BFS, DFS, and A* would provide a broader understanding of their relative strengths and weaknesses in the context of the Taxi-v3 environment and similar problem domains.

Environmental Realism and Complexity

The simplified and deterministic nature of the Taxi-v3 environment limits the generalizability of the findings to real-world scenarios. In real-world applications, pathfinding algorithms often encounter dynamic and unpredictable environments with factors such as traffic, obstacles, and changing conditions. Future research could enhance the complexity and realism of the environment by incorporating these dynamic elements. This would allow for a more robust evaluation of the algorithms' performance under

realistic conditions and provide insights into their adaptability and resilience to unforeseen changes.

User Guide

Software Requirements

- Python: 3.11.11 (Suitable Version: 3.7 to 3.11)
- IDE: Visual Studio Code
- Gymnasium
- Miniconda (32/64-bits) or Anaconda (32/64-bits)

To install Python and Gymnasium, the commands are as follows:

- Open Anaconda Prompt

To create the Gymnasium environment, run the following commands:

- conda create -n [environment name of your choice]
- conda activate [environment name of your choice]

To install Python, the commands are as follows:

- conda install python=3.11

To install Gymnasium, run the following commands:

- pip install gymnasium [classic-controls]
- pip install gymnasium [toy-text]
- pip install gymnasium [mujoco]
- pip install gymnasium [atari]
- pip install gymnasium [accept-rom-license]
- pip install gymnasium [box2d]

You may experience some difficulties when installing Gymnasium on Windows, so here are some instructions to guide you through it:

- conda install swig

- Head to https://visualstudio.microsoft.com/visual-cpp-build-tools/ and download Visual Studio Build Tools
- Install Desktop Development with C++ and close the application
- pip install gymnasium [box2d]